

10 - Condicionales y bucles en Javascript

Condicionales en JavaScript

Las condicionales son estructuras que permiten ejecutar una parte del código solo si se cumple una condición. Son esenciales para tomar decisiones en nuestros programas.

if - La condicional básica

La estructura básica es el if, que ejecuta un bloque de código si una condición es verdadera. La sintaxis es la siguiente:

```
if (condición) {  
    // Este bloque se ejecuta si la condición es verdadera  
}
```

Ejemplo

```
let edad = 18;  
  
if (edad >= 18) {  
    console.log("Eres mayor de edad");  
}
```

En este caso, si la variable edad es mayor o igual a 18, se imprime "Eres mayor de edad" en la consola. Si la condición es falsa, no pasa nada.

else - El caso contrario

¿Qué pasa si la condición no se cumple? Ahí entra else, que especifica qué hacer cuando la condición es falsa.

Sintaxis

```
if (condición) {  
    // Se ejecuta si la condición es verdadera  
} else {  
    // Se ejecuta si la condición es falsa  
}
```

Ejemplo

```
let edad = 16;  
  
if (edad >= 18) {  
    console.log("Eres mayor de edad");  
} else {  
    console.log("Eres menor de edad");  
}
```

Aquí, si la edad es menor a 18, se imprime "Eres menor de edad".

else if - Múltiples condiciones

A veces no basta con tener solo dos opciones, y quieres verificar varias condiciones. Aquí entra else if, que te permite agregar más casos entre el if y el else.

Sintaxis

```
if (condición1) {
```

```
// Se ejecuta si condición1 es verdadera
} else if (condición2) {
  // Se ejecuta si condición2 es verdadera
} else {
  // Se ejecuta si ninguna de las condiciones anteriores es verdadera
}
```

Ejemplo

```
let nota = 85;

if (nota >= 90) {
  console.log("Excelente");
} else if (nota >= 70) {
  console.log("Aprobado");
} else {
  console.log("Reprobado");
}
```

En este ejemplo, si nota es mayor o igual a 90, se imprime "Excelente"; si es mayor o igual a 70 pero menor a 90, imprime "Aprobado". Si ninguna de esas condiciones se cumple, imprime "Reprobado".

Operador ternario: la versión compacta del if-else

El operador ternario es una forma más corta de escribir un if-else. Su sintaxis es:

```
condición ? valorSiVerdadero : valorSiFalso;
```

Es útil para condicionales simples que devuelven un valor u otra cosa basada en una condición. No lo uses para condicionales complejas, porque puede volverse confuso.

Ejemplo básico

```
let edad = 20;
let mensaje = edad >= 18 ? "Eres mayor de edad" : "Eres menor de edad";
console.log(mensaje);
```

Aquí, si edad es mayor o igual a 18, la variable mensaje contendrá "Eres mayor de edad"; si no, contendrá "Eres menor de edad". Es como un if-else, pero todo en una línea.

Operadores Lógicos en Condicionales

Cuando trabajamos con condicionales en JavaScript, como los bloques if, a menudo necesitamos evaluar más de una condición. Es ahí donde entran en juego los operadores lógicos, que permiten combinar varias expresiones en una sola condición.

Los dos operadores lógicos más importantes que utilizamos en condicionales son:

1. AND (&&)

El operador AND (&&) evalúa si todas las condiciones son verdaderas. Solo si ambas condiciones son verdaderas, la expresión completa devolverá true. Si alguna de las condiciones es falsa, la expresión completa será false.

Sintaxis

```
condición1 && condición2
```

Ejemplo 1:

```
let edad = 25;
let tieneLicencia = true;

if (edad >= 18 && tieneLicencia) {
  console.log("Puedes conducir");
} else {
```

```
console.log("No puedes conducir");  
}
```

Explicación:

Aquí, solo si edad es mayor o igual a 18 y tieneLicencia es true, el mensaje "Puedes conducir" aparecerá. Si cualquiera de las dos condiciones es falsa (por ejemplo, tieneLicencia es false), el mensaje será "No puedes conducir".

Ejemplo 2:

```
let temperatura = 30;  
let esSoleado = true;  
  
if (temperatura > 20 && esSoleado) {  
    console.log("Es un buen día para salir al parque");  
} else {  
    console.log("Mejor quedarse en casa");  
}
```

Explicación:

Solo si temperatura es mayor a 20 y esSoleado es true, el mensaje "Es un buen día para salir al parque" se imprimirá en la consola.

2. OR (||)

El operador OR (||) evalúa si alguna de las condiciones es verdadera. Si una o ambas condiciones son verdaderas, la expresión completa devolverá true. Si ambas son falsas, la expresión será false.

Sintaxis

```
condición1 || condición2
```

Ejemplo 1:

```
let dia = "sábado";
let esVacaciones = false;

if (dia === "sábado" || dia === "domingo" || esVacaciones) {
  console.log("Hoy no hay trabajo");
} else {
  console.log("Hoy tienes que trabajar");
}
```

Explicación:

Si es sábado, domingo o es un día de vacaciones (esVacaciones es true), se mostrará "Hoy no hay trabajo". Si ninguna de estas condiciones es verdadera, el mensaje será "Hoy tienes que trabajar".

Ejemplo 2:

```
let estaLloviendo = false;
let tieneParaguas = true;

if (estaLloviendo || tieneParaguas) {
  console.log("Puedes salir sin problema");
} else {
  console.log("Mejor quédate en casa");
}
```

Explicación:

Aquí, si está lloviendo o la persona tiene un paraguas, el mensaje "Puedes salir sin problema" se imprimirá. Si ninguna de estas dos condiciones es verdadera (no está lloviendo y no tiene paraguas), aparecerá el mensaje "Mejor quédate en casa".

Operadores Lógicos en Condicionales: AND y OR Combinados

Podemos combinar tanto el operador AND (&&) como el OR (||) en una misma condición para obtener expresiones más complejas.

Ejemplo:

```
let esFinDeSemana = true;
let haceSol = false;
let tienesTiempoLibre = true;

if ((esFinDeSemana || tienesTiempoLibre) && haceSol) {
  console.log("Podemos ir a la playa");
} else {
  console.log("Mejor hacemos otra cosa");
}
```

Explicación:

Este ejemplo es un poco más avanzado, pero sigue siendo claro. El código verificará si es fin de semana o si tienes tiempo libre. Si al menos una de esas condiciones es verdadera y hace sol, entonces se mostrará "Podemos ir a la playa". Si no, se imprimirá "Mejor hacemos otra cosa".

Ejercicio con Operadores Lógicos

Ejercicio 1:

Escribe un programa que le pida al usuario su edad y si tiene una licencia de conducir. Si la edad es mayor o igual a 18 y tiene licencia, debe mostrar el mensaje "Puedes conducir". Si alguna de las dos condiciones no se cumple, debe mostrar el mensaje "No puedes conducir".

Código

```
let edad = prompt("¿Cuántos años tienes?");
let tieneLicencia = prompt("¿Tienes licencia de conducir? (sí o no)");

if (edad >= 18 && tieneLicencia === "sí") {
  alert("Puedes conducir");
} else {
```

```
alert("No puedes conducir");  
}
```

Ejercicio 2:

Escribe un programa que le pregunte al usuario si está lloviendo y si tiene paraguas. Si está lloviendo o tiene paraguas, debe mostrar el mensaje "Puedes salir". Si no se cumple ninguna de las dos, el programa mostrará "Mejor quédate en casa".

Código

```
let estaLloviendo = prompt("¿Está lloviendo? (sí o no)") === "sí";  
let tieneParaguas = prompt("¿Tienes paraguas? (sí o no)") ===  
"sí";  
  
if (estaLloviendo || tieneParaguas) {  
  alert("Puedes salir");  
} else {  
  alert("Mejor quédate en casa");  
}
```

El uso de los operadores lógicos AND (&&) y OR (||) es crucial para crear condiciones más precisas y complejas en nuestros programas. Nos permiten evaluar múltiples situaciones y tomar decisiones más inteligentes. Practicar su uso te ayudará a escribir código más flexible y dinámico, que reaccione de manera adecuada a diferentes escenarios.

¡Recuerda siempre que estos operadores hacen que los condicionales sean mucho más potentes y adaptables!

Bucles en JavaScript

Los bucles permiten ejecutar un bloque de código varias veces. Son muy útiles cuando necesitas repetir una tarea varias veces, como mostrar una lista de elementos, contar números, o procesar datos.

Bucle for

El bucle for es ideal cuando sabes cuántas veces necesitas repetir un bloque de código. Es un bucle controlado por un contador, lo que significa que tiene tres partes esenciales: una inicialización, una condición de continuación, y una actualización.

Sintaxis del for

```
for (inicialización; condición; actualización) {  
    // Código a ejecutar en cada iteración  
}
```

Inicialización: Se ejecuta una sola vez al principio y establece el valor inicial de una variable de control (por lo general i).

Condición: Mientras esta condición sea verdadera, el bucle sigue ejecutándose.

Actualización: Se ejecuta al final de cada iteración y suele incrementar o cambiar la variable de control.

Ejemplo

```
for (let i = 0; i < 5; i++) {  
    console.log(`Iteración número ${i}`);  
}
```

Este bucle imprimirá:

```
Iteración número 0  
Iteración número 1  
Iteración número 2  
Iteración número 3  
Iteración número 4
```

El contador *i* comienza en 0, y el bucle se ejecuta mientras *i* sea menor que 5. En cada iteración, *i* se incrementa en 1.

Bucle while

El bucle while es útil cuando no sabes exactamente cuántas veces necesitas repetir algo, pero tienes una condición que debe cumplirse para continuar.

Sintaxis del while

```
while (condición) {  
    // Código a ejecutar mientras la condición sea verdadera  
}
```

La condición se verifica antes de cada iteración. Si es falsa, el bucle no se ejecuta más.

Ejemplo

```
let contador = 0;  
  
while (contador < 3) {  
    console.log(`Contador: ${contador}`);  
    contador++;  
}
```

Este bucle imprimirá:

```
Contador: 0  
Contador: 1  
Contador: 2
```

El código sigue ejecutándose mientras contador sea menor que 3, y se incrementa con cada iteración.

Bucle do...while

El bucle do...while es similar al while, pero con una diferencia clave: el bloque de código se ejecuta al menos una vez, porque la condición se verifica después de la primera iteración.

Sintaxis del do...while

```
do {  
  // Código a ejecutar al menos una vez  
} while (condición);
```

Ejemplo

```
let num = 0;  
do {  
  console.log(`Número: ${num}`);  
  num++;  
} while (num < 2);
```

Este bucle imprimirá:

```
Número: 0  
Número: 1
```

Incluso si la condición es falsa desde el principio, el código dentro del bloque do se ejecutará una vez.

Ejercicios prácticos

Ejercicio 1: Condicional básica

Escribe un código que verifique si una persona es mayor de edad. Si tiene 18 años o más, imprime "Mayor de edad", de lo contrario, imprime "Menor de edad".

```
let edad = 17;

if (edad >= 18) {
  console.log("Mayor de edad");
} else {
  console.log("Menor de edad");
}
```

Ejercicio 2: Bucle for para contar

Usa un bucle for para imprimir los números del 1 al 10.

```
for (let i = 1; i <= 10; i++) {
  console.log(i);
}
```

Ejercicio 3: Bucle while con un contador

Escribe un bucle while que cuente del 1 al 5.

```
let contador = 1;

while (contador <= 5) {
  console.log(contador);
  contador++;
}
```

Ejercicio 4: Operador ternario

Usa el operador ternario para verificar si una persona tiene la edad suficiente para votar (18 o más). Si es así, imprime "Puede votar", si no, imprime "No puede votar".

```
let edad = 16;
let mensaje = edad >= 18 ? "Puede votar" : "No puede votar";
console.log(mensaje);
```

Ejercicio 5: Bucle do...while

Crea un bucle do...while que imprima un número inicial y lo incremente hasta que sea mayor a 3.

```
let numero = 0;

do {
  console.log(`Número: ${numero}`);
  numero++;
} while (numero <= 3);
```

Conclusión

Ahora tienes una comprensión sólida de cómo usar condicionales y bucles en JavaScript. Estas estructuras te permiten controlar el flujo de tu programa de manera eficiente. En los próximos capítulos, profundizaremos en más conceptos, pero por ahora sigue practicando con estas herramientas fundamentales!